

Todaily Realtime Architecture Guide

Complete Fullstack Engineering Plan for Chat, Voice, Video, Docker, Redis, Mediasoup & Service Separation

Technologies Included	Next.js • Express • PostgreSQL • Drizzle • Redis • Socket.IO • Mediasoup • Coturn
-----------------------	---

You Are Moving Beyond CRUD

You are moving from a traditional MERN CRUD developer into systems-oriented realtime engineering. This architecture teaches distributed processes, realtime state synchronization, service separation, WebRTC, media routing, Redis Pub/Sub, and scalable socket systems.

Separate Service $\neq$ Separate Database

Your services can share one PostgreSQL database, one Redis instance, one Docker network, and shared TypeScript packages while still being separate Node.js processes.

Todaily Final Architecture

```
NGINX
  ↓
Frontend (Next.js)

API Service
Realtime Service
RTC Service

Shared Infrastructure:
- PostgreSQL
- Redis
- Docker Network
```

API Service Responsibilities

```
Handles:
- Auth
- Workspace
- Projects
- Tasks
- Permissions
- Upload Metadata
```

```
Important:
NO SOCKETS HERE.
This service owns business logic.
```

Realtime Service Responsibilities

Handles:

- Chat messages
- Presence
- Typing indicators
- Notifications
- Read receipts
- Unread synchronization

This service is socket-heavy.

RTC Service Responsibilities

Handles:

- Mediasoup
- WebRTC signaling
- Voice rooms
- Video rooms
- Transports
- Producers
- Consumers
- Room coordination

This service is realtime-heavy.

How Services Communicate

Docker creates an internal network.

Services communicate internally like:

```
http://api:5000
http://realtime:5001
http://rtc:5002
```

NOT localhost.

Authentication Flow

Frontend

↓

RTC Service
↓
Verify JWT
↓
Ask API service for project/workspace access
↓
Allow or reject room join

Recommended Monorepo Structure

```
todaily/  
  
apps/  
  web/  
  api/  
  realtime/  
  rtc/  
  
packages/  
  db/  
  shared/  
  socket-events/  
  types/  
  
docker/  
  nginx/  
  redis/  
  postgres/  
  coturn/
```

Redis Usage

Use Redis For:

- Socket.IO adapter
- Presence
- Pub/Sub
- Realtime synchronization
- RTC participant state
- Scaling websocket instances

Do NOT use Redis for permanent chat history.

Chat System Plan

Step 1:
Create chat tables.

Step 2:
Create realtime socket gateway.

Step 3:
Implement:
- room joining
- send message
- typing indicators
- read receipts

Step 4:
Add Redis adapter.

Step 5:
Add optimistic UI updates.

Chat Database Design

Tables:
- chat_rooms
- messages
- room_members
- message_mentions
- message_attachments

Important:
Use REST APIs for history/pagination.
Use sockets only for realtime updates.

Voice System Plan

Step 1:
Learn basic WebRTC:
- offer
- answer
- ICE

Step 2:
Integrate Mediasoup.

Step 3:
Create:
- workers
- routers
- transports

Step 4:
Create producer/consumer flow.

Step 5:
Implement room synchronization.

RTC Flow

```
graph TD
    A[join room] --> B[create transport]
    B --> C[connect transport]
    C --> D[produce audio]
    D --> E[consume others]
```

Video System Plan

Video extends voice infrastructure.

Features:

- camera toggle
- screen share
- participant grid
- active speaker
- reconnect logic

Start with:
720p + 15fps

Important Video Optimization

Do NOT send high-quality video to every participant.

Use:

- simulcast
- selective consumers
- visible participant prioritization

Mediasoup supports this.

Socket Architecture

Namespaces:

```
/chat  
/rtc
```

This prevents:

- event collisions
- noisy realtime architecture

Docker Services

```
frontend  
api  
realtime  
rtc  
postgres  
redis  
nginx  
coturn
```

Recommended VPS

Minimum:

```
4 vCPU  
8 GB RAM
```

Better:

```
8 vCPU  
16 GB RAM
```

Especially important for video rooms.

Final Engineering Advice

Do NOT overengineer.

You do NOT need:

- Kafka
- Kubernetes
- 20 microservices

Final Technology Stack Summary

Layer	Technology
Frontend	Next.js
API	Express + Drizzle + PostgreSQL
Realtime	Socket.IO + Redis
RTC	Mediasoup + WebRTC
Infra	Docker + NGINX + Coturn